



程序设计原理

变量声明与基本运算

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



Constants

const datatype CONSTANTNAME = VALUE;

A constant must be declared and initialized in the same statement

```
#include <iostream>
using namespace std;

int main()
{
    const int LENGTH = 10;
    const int WIDTH = 5;
    const char NEWLINE = '\n';
    int area;

    area = LENGTH * WIDTH;
    cout << area;
    cout << NEWLINE;
    return 0;
}
```



Constants

#define identifier value

```
#include <iostream>
using namespace std;

#define LENGTH 10
#define WIDTH 5
#define NEWLINE '\n'

int main()
{

    int area;

    area = LENGTH * WIDTH;
    cout << area;
    cout << NEWLINE;
    return 0;
}
```



Numerical Data Types and Operations

Name	Synonymy	Range	Storage Size
short	short int	-2^{15} (-32,768) to $2^{15}-1$ (32,767)	16-bit signed
unsigned short	unsigned short int	0 to $2^{16}-1$ (65535)	16-bit unsigned
int		-2^{31} (-2147483648) to $2^{31}-1$ (2147483647)	32-bit signed
unsigned int	unsigned	0 to $2^{32}-1$ (4294967295)	32-bit unsigned
long	long int	-2^{31} (-2147483648) to $2^{31}-1$ (2147483647)	32-bit signed
unsigned long	unsigned long int	0 to $2^{32}-1$ (4294967295)	32-bit unsigned
float		Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
double		Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754
long double		Negative range: -1.18E+4932 to 3.37E-4932 Positive range: 3.37E-4932 to 1.18E+4932 Significant decimal digits: 19	80-bit



Overflow

When a variable is assigned a value that is too large to be stored, it causes *overflow*. For example, executing the following statement causes *overflow*, because the largest value that can be stored in a variable of the short type is 32767. 32768 is too large.

short value = 32767 + 1;



Underflow

When a variable is assigned a value that is too small to be stored, it causes *underflow*. For example, executing the following statement causes *underflow*, because the smallest value that can be stored in a variable of the short type is -32768. -32769 is too small.

short value = -32769;



Numeric Operators

Name	Meaning	Example	Result
+	Addition	$34 + 1$	35
-	Subtraction	$34.0 - 0.1$	33.9
*	Multiplication	$300 * 30$	9000
/	Division	$1.0 / 2.0$	0.5
%	Remainder	$20 \% 3$	2



Integer Division

$+$, $-$, $*$, $/$, and $\%$

$5 / 2$ yields an integer 2.

$5.0 / 2$ yields a double value 2.5

$5 \% 2$ yields 1 (the remainder of the division)



Remainder Operator

Remainder is very useful in programming. For example, an even number % 2 is always 0 and an odd number % 2 is always 1. So you can use this property to determine whether a number is even or odd. Suppose today is Saturday and you and your friends are going to meet in 10 days. What day is in 10 days? You can find that day is Tuesday using the following expression:

Saturday is the 6th day in a week

$(6 + 10) \% 7$ is 2

A week has 7 days

The 2nd day in a week is Tuesday

After 10 days



Formatting Console Output

```
double amount = 12618.98;  
double interestRate = 0.0013;  
double interest = amount * interestRate;
```

```
cout << "Interest is:"<<interest<<endl;  
Interest is: 16.4047
```

```
cout << "Interest is:"  
<<fixed<<setprecision(2)<<interest<<endl;  
Interest is: 16.40
```



Conversion Rules

When performing a binary operation involving two operands of different types, C++ automatically converts the operand based on the following rules:

1. If one of the operands is long double, the other is converted into long double.
2. Otherwise, if one of the operands is double, the other is converted into double.
3. Otherwise, if one of the operands is float, the other is converted into float.
4. Otherwise, if one of the operands is unsigned long, the other is converted into unsigned long.
5. Otherwise, if one of the operands is long, the other is converted into long.
6. Otherwise, if one of the operands is unsigned int, the other is converted into unsigned int.
7. Otherwise, both operands are converted into int.



Type Casting

Implicit casting

`double d = 3;` (type widening)

Explicit casting

`int i = static_cast<int>(3.0);` (type narrowing)

`int i = (int)3.9;` (Fraction part is truncated)

Casting does not change the variable being cast. For example, d is not changed after casting in the following code:

`double d = 4.5;`

`int i = static_cast<int>(d);` // d is not changed



Augmented Assignment Operators

<i>Operator</i>	<i>Example</i>	<i>Equivalent</i>
<code>+=</code>	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	<code>f -= 8.0</code>	<code>f = f - 8.0</code>
<code>*=</code>	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	<code>i /= 8</code>	<code>i = i / 8</code>
<code>%=</code>	<code>i %= 8</code>	<code>i = i % 8</code>



Increment and Decrement Operators

Operator	Name	Description
<u>++var</u>	preincrement	The expression (++var) increments <u>var</u> by 1 and evaluates to the <i>new</i> value in <u>var</u> <i>after</i> the ncrement.
<u>var++</u>	postincrement	The expression (var++) evaluates to the <i>original</i> value in <u>var</u> and increments <u>var</u> by 1.
<u>--var</u>	predecrement	The expression (--var) decrements <u>var</u> by 1 and evaluates to the <i>new</i> value in <u>var</u> <i>after</i> the ecrement.
<u>var--</u>	postdecrement	The expression (var--) evaluates to the <i>original</i> value in <u>var</u> and decrements <u>var</u> by 1.



Increment and Decrement Operators

```
int i = 10;  
int newNum = 10 * i++;
```

```
int i = 10;  
int newNum = 10 * (++i);
```



Increment and Decrement Operators

Using increment and decrement operators makes expressions short, but it also makes them complex and difficult to read.

Avoid using these operators in expressions that modify multiple variables, or the same variable for multiple times such as this: int k = ++i + i. (Undefined behavior in the standard)



Exercise

A/B Problem

Write a program which reads two integers a and b , and calculates the following values:

$a \div b$: d (in integer)

remainder of $a \div b$: r (in integer)

$a \div b$: f (in real number)

Input

Two integers a and b are given in a line.

Output

Print d , r and f separated by a space in a line. For f , the output should not contain an absolute error greater than 10^{-5} .

Constraints

$1 \leq a, b \leq 10^9$

Sample Input 1

3 2

Sample Output 1

1 1 1.50000



Exercise

X Cubic

Write a program which calculates the cube of a given integer x.

Input

An integer x is given in a line.

Output

Print the cube of x in a line.

Constraints

$$1 \leq x \leq 100$$

Sample Input 1

2

Sample Output 1

8

Sample Input 2

3

Sample Output 2

27



Exercise

Rectangle

Write a program which calculates the area and perimeter of a given rectangle.

Input

The length a and breadth b of the rectangle are given in a line separated by a single space.

Output

Print the area and perimeter of the rectangle in a line. The two integers should be separated by a single space.

Constraints

$$1 \leq a, b \leq 100$$

Sample Input 1

3 5

Sample Output 1

15 16



Exercise

Watch

Write a program which reads an integer S [second] and converts it to $h:m:s$ where h, m, s denote hours, minutes (less than 60) and seconds (less than 60) respectively.

Input

An integer S is given in a line.

Output

Print h, m and s separated by ':'.
You do not need to put '0' for a value, which consists of a digit.

Constraints

$$0 \leq S \leq 86400$$

Sample Input 1

46979

Sample Output 1

13:2:59



Exercise

Circle

Write a program which calculates the area and circumference of a circle for given radius r .

Input

A real number r is given.

Output

Print the area and circumference of the circle in a line. Put a single space between them. The output should not contain an absolute error greater than 10^{-5} .

Constraints

$0 < r < 10000$

Sample Input 1

2

Sample Output 1

12.566371 12.566371

Sample Input 2

3

Sample Output 2

28.274334 18.849556